

Group Project Introduction

Introduction

This group project is the final coursework component of the course this semester, which accounts for **40%** of your final mark. You will form a **team of two** with your classmates and apply object-oriented principles and advanced data structures, as learned throughout the semester, to solve the tasks specified in this document.

You will be required to write a report and create a video presentation to demonstrate your understanding of:

1. Object-oriented concepts and software design;
2. Problem-solving techniques using sorting and graph algorithms;
3. AI-assisted team cooperation and project management;
4. Equality, diversity, and inclusion (EDI) principles;
5. Planning, reflection, and self-learning in object-oriented programming.

You are required to submit the following files (see Submission Requirement Section for more details):

- Java code in a ZIP file
- A Word report
- An MP4 video
- A PowerPoint (PPT) presentation used in the video

Stage	Date
Task sheet release	Week 9, Monday, 27th of April, 9:00 am (UTC+8)
Submission	Week 12, Sunday, 24th of May, 11:59 pm (UTC+8)
Submission cut-off	Week 13, Friday, 29th of May, 11:59 pm (UTC+8) <i>There will be 5% lateness penalty per working day, up to a maximum of 5 working days. (Monday-Friday, Week 13). No submissions are accepted after 11:59 pm (UTC+8), 29th of May.</i>

- **Task Sheet will be released on Monday, Week 9**
- **40%** of the final mark
- Team of **2** (Please do so by today)
- **4 files** need to be submitted
- From Apr 27 to May 24; **DDL** - May 24, 11:59 p.m.

Tasks Overview

Project Brief

Scenario

A city plans to carry out a large-scale infrastructure inspection task. Three different information sources each provide a candidate list of locations that may require inspection. Since these datasets come from different systems and processing workflows, they have different properties (e.g., ordered or unordered). Your program is required to process these datasets, identify important inspection targets, and then compute routes for visiting them efficiently.

Overview

In this coursework, you are required to **design and implement an object-oriented Java application** for an **Urban Infrastructure Inspection System**, integrating appropriate data structures, sorting algorithms, graph algorithms, and object-oriented design principles.

The coursework consists of five tasks:

- **Task A** requires you to analyze the behaviour and performance of three sorting algorithms on the three given candidate datasets, and to select the top 10 highest-priority locations from each dataset.
- **Task B** requires you to use graph algorithms to compute route or path results for visiting the selected inspection targets (from Task A) on a weighted graph.
- **Task C** requires you to explain and justify the design of your overall application, with a particular focus on data structures, class design, and object-oriented programming principles.
- **Task D** requires you to reflect on your project process, including AI-assisted planning, teamwork, and equality, diversity, and inclusion considerations.
- **Task E** requires you to present your project in the form of a PPT and a video explanation.

- **Scenario:** An inspection task of the infrastructures in a city (e.g., road/traffic lights, 5G stations, etc)

Task A: Use sorting algorithms (i.e., bubble ,quick, merge sort) to find out the highest-priority locations, and evaluate the algorithms' performances.

	A	B
1	location	priority_score
2	L0337	9664
3	L0513	9488
4	L0394	9607
5	L0969	9032
6	L0171	9830
7	L0082	9909

Task B: Plan the path/route visiting the selected locations using Graph algorithms. Justify and evaluate the algorithms.

Task C: Description of your program (e.g., how the OOP principles are applied)

Task D: A project reflection (e.g., how the AI Management Tools like JIRA/KANBAN are used for planning)

Task E: An 8-minute video presentation with PPT.

General Notes

Report Requirements

Structure

The purpose of your report is to show how you solved **Tasks A, B, C, and D** in a well-detailed manner. You need to organize your report in the following format:

- Coursework submission cover page
(This page is **compulsory**. You can download the template from the CPT204 LMO homepage if required.)
- Chapter 1 – Sorting Algorithm (Task A)
 - (You may add sub-chapters)
- Chapter 2 – Graph Algorithm (Task B)
 - (You may add sub-chapters)
- Chapter 3 – Design of the Overall Application (Task C)
 - (You may add sub-chapters)
- Chapter 4 – Project Reflection (Task D)
 - (You may add sub-chapters)
- Chapter 5 – Program Code
 - Include all your code in Chapter 5 as text. Copy and paste each source file content into the report. **The use of screenshots in Chapter 5 should not be accepted.**
- Chapter 6 – Appendix
 - You may use this section to include any supplementary sources (e.g., console outputs, extra sorting results, shortest-path outputs, UML diagrams not included in the main text).
- Chapter 7 – Contribution Form
 - Use the following table to indicate your contribution:

20 pages max

Student ID	Contribution

*Note: The total percentage is **100%**. That is, if you and your teammate contribute equally to this project, both of you should fill **50%** in the table.*

- Sign and include the **Coursework submission cover page** in your report (download from LMO CPT204 homepage)
- Report - 20-page limitation (for Ch.1-4 only)
- Copy and paste your code in Chapter 5, do **NOT** use screenshots
- Contribution form - You are encouraged to contribute equally to this project (i.e., 50% and 50%)
- **DO NOT USE AI FOR CODING/ANSWER GENERATION**
- Support:
 - Monday 3-5pm, Tuesday 1-3pm, SD423
 - Better in-person, extra time outside the office hour is available by appointment.

Sorting

CPT 204 - Advanced OO
Programming

AY 25/26

Objectives

- To design, implement, and analyze *bubble sort*
- To design, implement, and analyze *merge sort*
- To design, implement, and analyze *quick sort*
- To design and implement a *binary heap*
- To design, implement, and analyze *heap sort*
- To study and analyze time complexity of the sorting algorithms above

Bubble Sort

- Bubble Sort uses **a nested for loop** to sort elements.
- **The outer for loop** controls how many **passes** are made through the array
- In each pass, **the inner for loop compares** each pair of adjacent elements **and swaps** them if they are in the wrong order.
- After each pass, the largest unsorted element will be sorted to the end of the array.

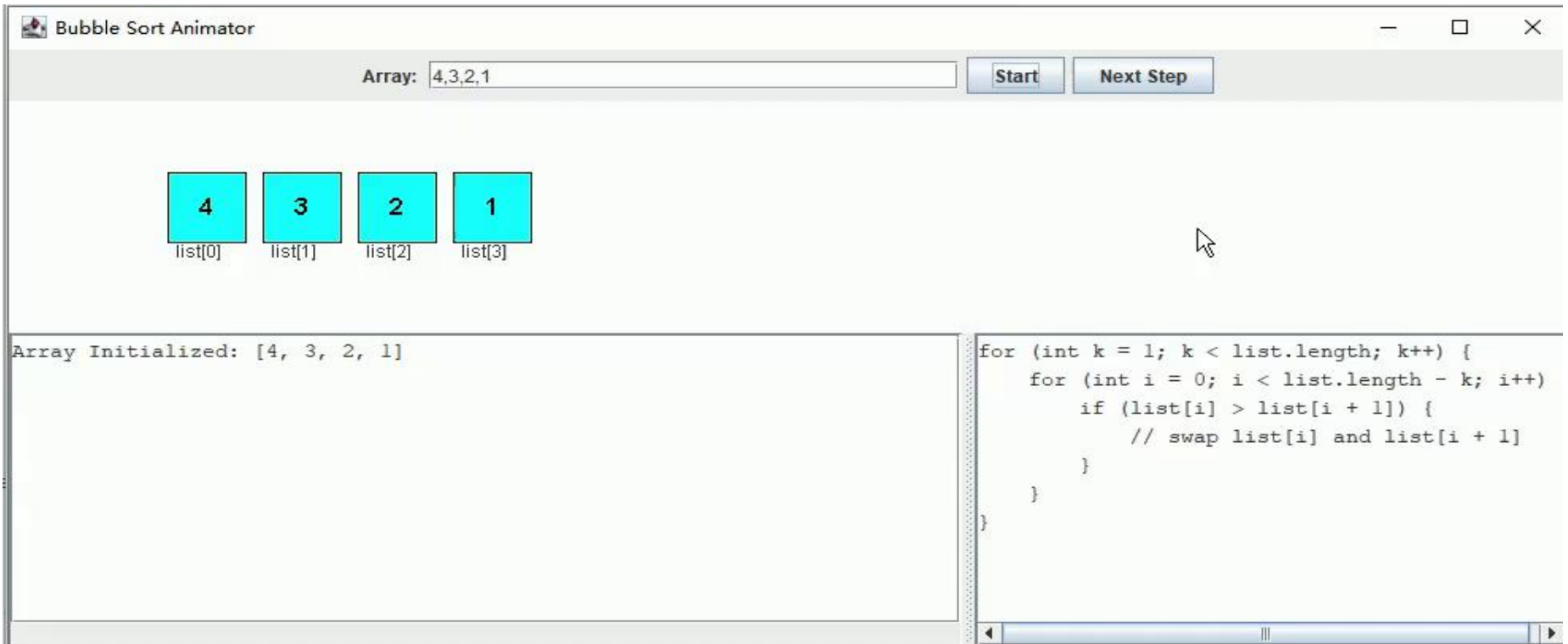
```
// Outer loop controls the number of passes through the
// entire array
for (int k = 1; k < list.length; k++) {

    // In each pass, the inner loop compares each pair of
    // adjacent elements
    for (int i = 0; i < list.length - k; i++) {
        if (list[i] > list[i + 1]){
            // swap list[i] with list[i + 1]
        }
    }
}
```



Because after k times passes, k elements will be sorted at the end, so the inner loop stops before that sorted part

Bubble Sort

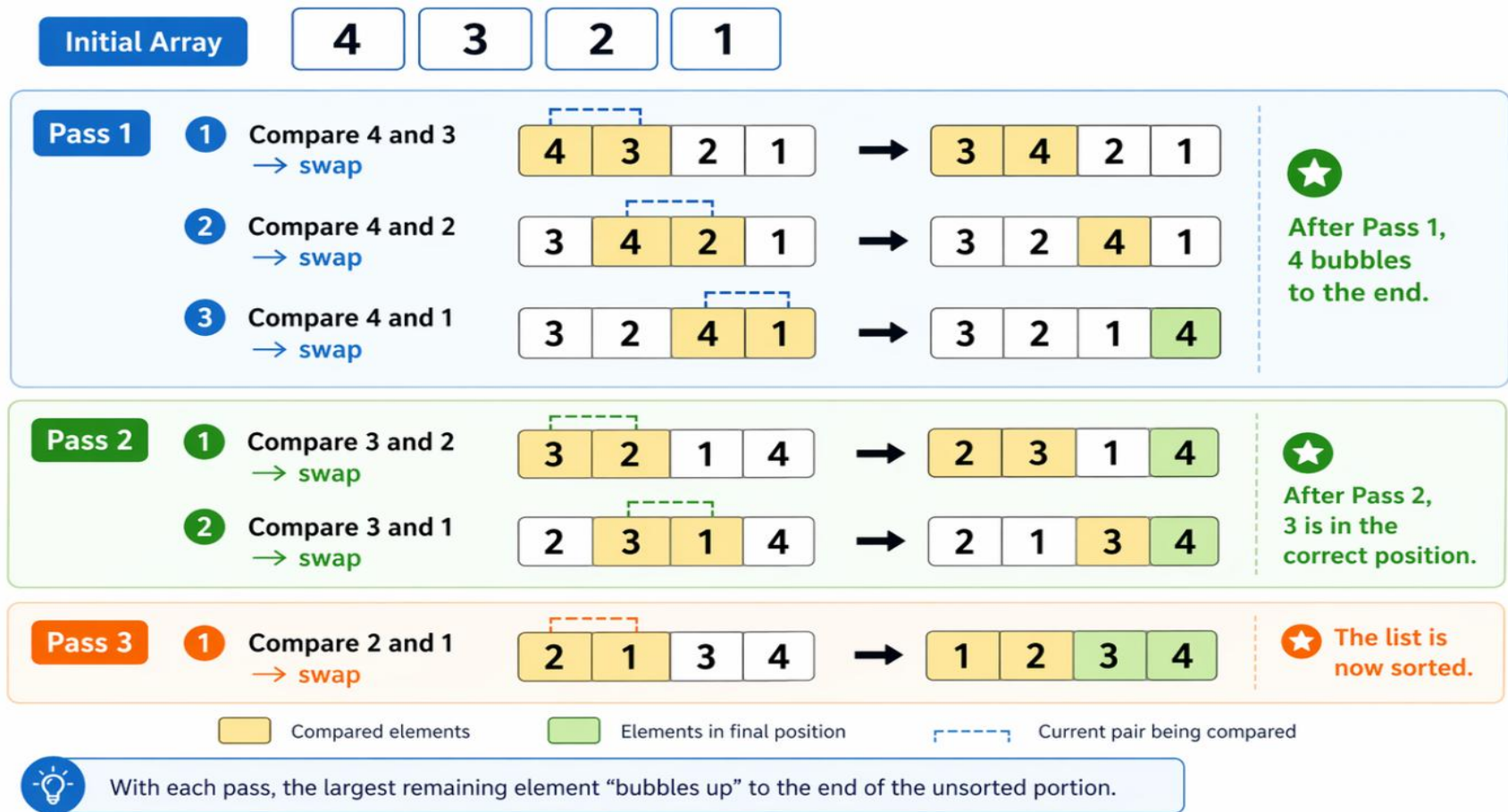


The screenshot shows a window titled "Bubble Sort Animator". At the top, there is a text input field labeled "Array:" containing the values "4,3,2,1". To the right of the input field are two buttons: "Start" and "Next Step". Below the input field, four cyan-colored boxes represent the array elements, each with a black border and a black outline. The boxes contain the numbers 4, 3, 2, and 1 from left to right. Below each box is a label: "list[0]" under 4, "list[1]" under 3, "list[2]" under 2, and "list[3]" under 1. A mouse cursor is visible to the right of the boxes. At the bottom of the window, there are two panels. The left panel is labeled "Array Initialized: [4, 3, 2, 1]" and is empty. The right panel contains a code editor with the following Java code:

```
for (int k = 1; k < list.length; k++) {  
    for (int i = 0; i < list.length - k; i++)  
        if (list[i] > list[i + 1]) {  
            // swap list[i] and list[i + 1]  
        }  
}
```

The code editor has a scrollbar on the right side.

Visualizing Bubble Sort Process



- In the example, for 4 elements, the number of comparisons are $4-1=3$ times (1st pass), $4-2=2$ times (2nd pass) and, $4-3=1$ time (3rd pass).
- So, for n elements, we compare $(n-1) + (n-2) + \dots + 2 + 1 = n*(n-1) / 2$ times.
- The Big(O) is then $O(n^2)$.

```

public class BubbleSort {
    public static void bubbleSort(int[] list) {
        for (int k = 1; k < list.length; k++) {
            for (int i = 0; i < list.length - k; i++) {
                if (list[i] > list[i + 1]) {
                    int temp = list[i];
                    list[i] = list[i + 1];
                    list[i + 1] = temp;
                }
            }
        }
    }
    public static void main(String[] args) {
        int size = 100000;
        int[] a = new int[size];
        randomInitiate(a);
        long startTime = System.currentTimeMillis();
        bubbleSort(a);
        long endTime = System.currentTimeMillis();
        System.out.println((endTime - startTime) + "ms");
    }
    private static void randomInitiate(int[] a) {
        for (int i = 0; i < a.length; i++)
            a[i] = (int) (Math.random() * a.length);
    }
}

```

14650ms

For list like [4,3,2,1], swaps are needed in every pass.

In the best case, if a list is almost or already ascending ordered like [1,2,3,4], what is the issue?

- There is **no swap is actually needed** in any of the passes.
- But a typical bubble sort would **keep doing passes** as long as $k < \text{list.length}$...wasting of time.

Questions:

- How can we optimize the current bubble sort to make it stop earlier?
- What is the Big O in this best case?

Hint: Use a **boolean variable** to check whether any swap happens during a pass. If not, stop the algorithm (try lab question Q2).

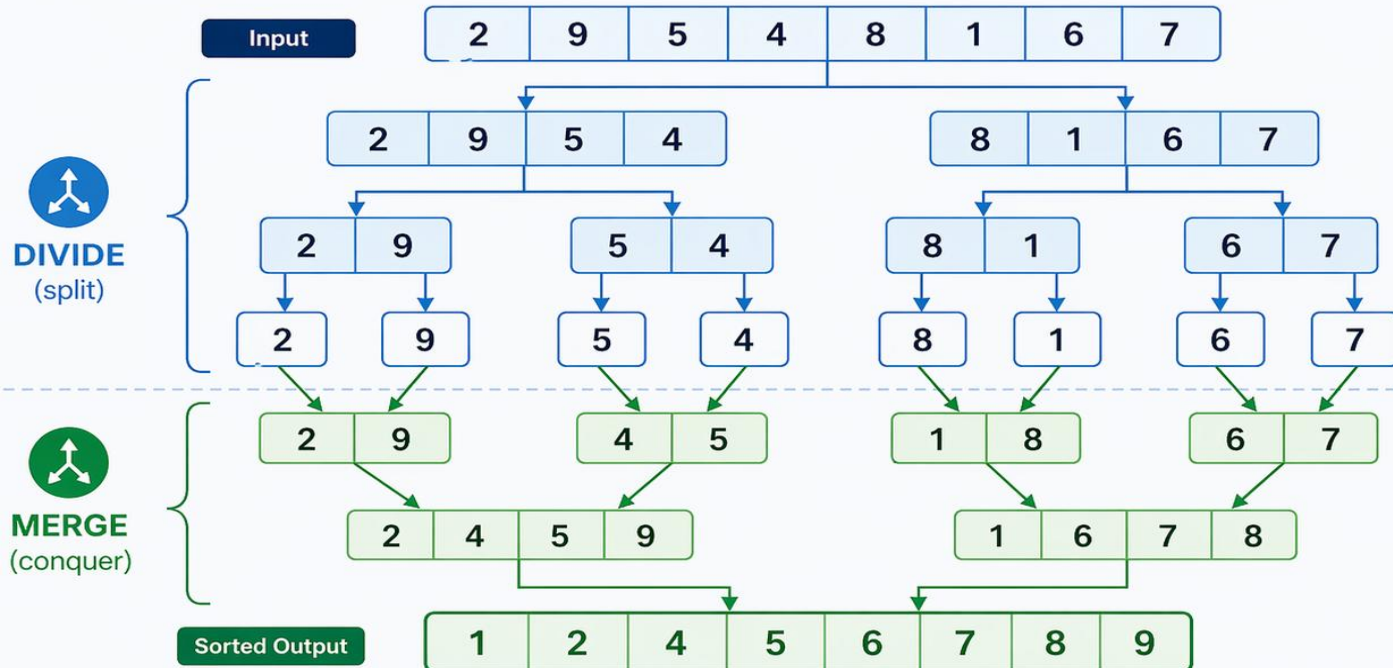
Merge Sort

- **Divide** the array, recursively, into two halves.
- Stop when each subarray has only one element (i.e., the base case).
- **Merge** the subarrays back together in sorted order.

- **Divide:** Split the array into smaller halves until each subarray contains just 1 element.
- **Merge:** Combine the subarrays back together in sorted order.



Base case:
stop dividing when
the subarray size is 1.



Dividing Process

```
public class MergeSortTest {  
    public static void mergeSort(int[] list) {  
        if (list.length > 1) {  
            // Dividing the first half  
            int[] firstHalf = new int[list.length / 2];  
            System.arraycopy(list, 0, firstHalf, 0, list.length  
                / 2);  
            // A recursive process as the mergeSort() is calling  
            itself  
            mergeSort(firstHalf);  
            // Dividing the second half  
            int secondHalfLength = list.length - list.length / 2;  
            int[] secondHalf = new int[secondHalfLength];  
            System.arraycopy(list, list.length / 2, secondHalf,  
                0, secondHalfLength);  
            // A recursive process as the mergeSort() is calling  
            itself  
            mergeSort(secondHalf);  
  
            merge(firstHalf, secondHalf, list);  
        }  
    }  
}
```

When the mergeSort (the recursion call) returns, the merge() can combine the halves level by level by level.

Para	Def
list	Source list
0	Start copying from
firsthalf	Destination list
0	Start placing to
list.length/2	Number of copied elements

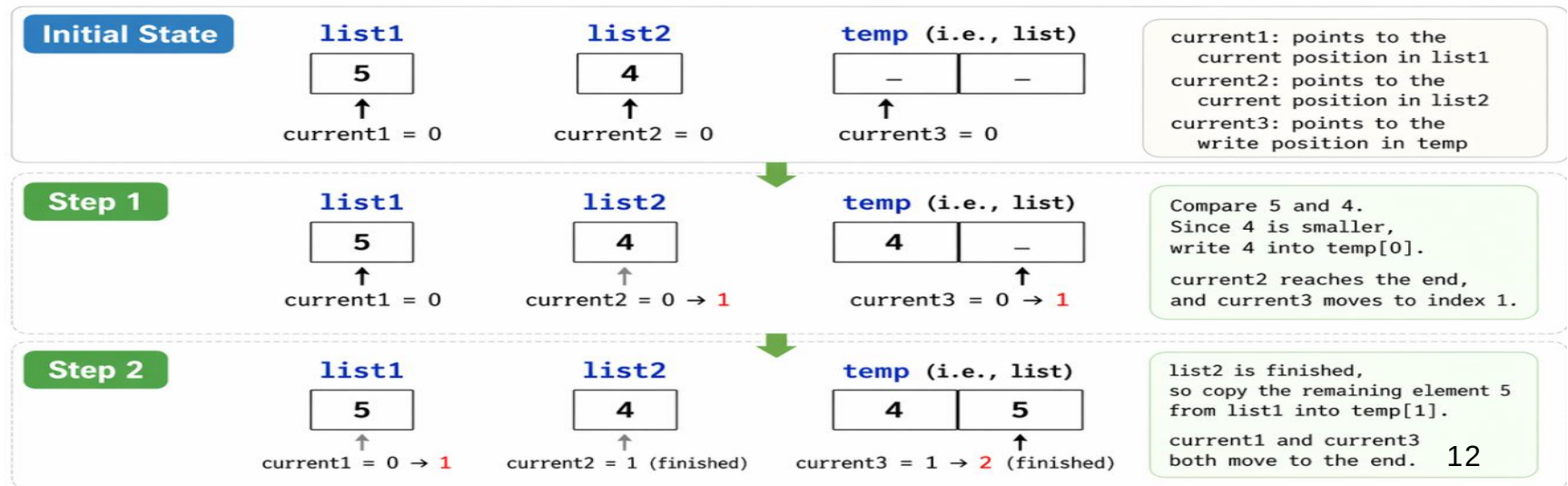
Merging Process

```
public static void merge(int[] list1, int[] list2, int[] temp){
    int current1 = 0; // Current index in list1, the 1st half
    int current2 = 0; // Current index in list2, the 2nd half
    int current3 = 0; // Current index in temp, storing data temporarily

    // Case1: Both of the sublists have remaining elements
    while (current1 < list1.length && current2 < list2.length) {
        // If current element in list1 is smaller, add it to temp, and the current1 and current3
        // indices/pointers move forward (because of the ++)
        if (list1[current1] < list2[current2])
            temp[current3++] = list1[current1++];
        else
            // Otherwise, add the current element in list2 to temp, and the current2 and current3
            // indices/pointers move forward (because of the ++)
            temp[current3++] = list2[current2++];
    }

    // Case2: List2 is finished, but there are remaining elements in list1, add them to temp
    while (current1 < list1.length)
        temp[current3++] = list1[current1++];

    // Case3: list1 is finished, but there are remaining elements in list2, add them to temp
    while (current2 < list2.length)
        temp[current3++] = list2[current2++];
}
```



Merge Sort Animator

List1: 2,4,5,9 List2: 1,6,7,8 Start Next Step Merge Started Code Window Height:

current1

2	4	5	9
---	---	---	---

current2

1	6	7	8
---	---	---	---

current3

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

List1: [2, 4, 5, 9]
List2: [1, 6, 7, 8]

```
public static void merge(int[] list1, int[] list2, int[] temp) {  
    int current1 = 0; // Current index in list1  
    int current2 = 0; // Current index in list2  
    int current3 = 0; // Current index in temp  
    while (current1 < list1.length && current2 < list2.length) {  
        if (list1[current1] < list2[current2])  
            temp[current3++] = list1[current1++];  
        else  
            temp[current3++] = list2[current2++];  
    }  
    while (current1 < list1.length)  
        temp[current3++] = list1[current1++];  
    while (current2 < list2.length)  
        temp[current3++] = list2[current2++];  
}
```

Visualizing a middle step with more elements

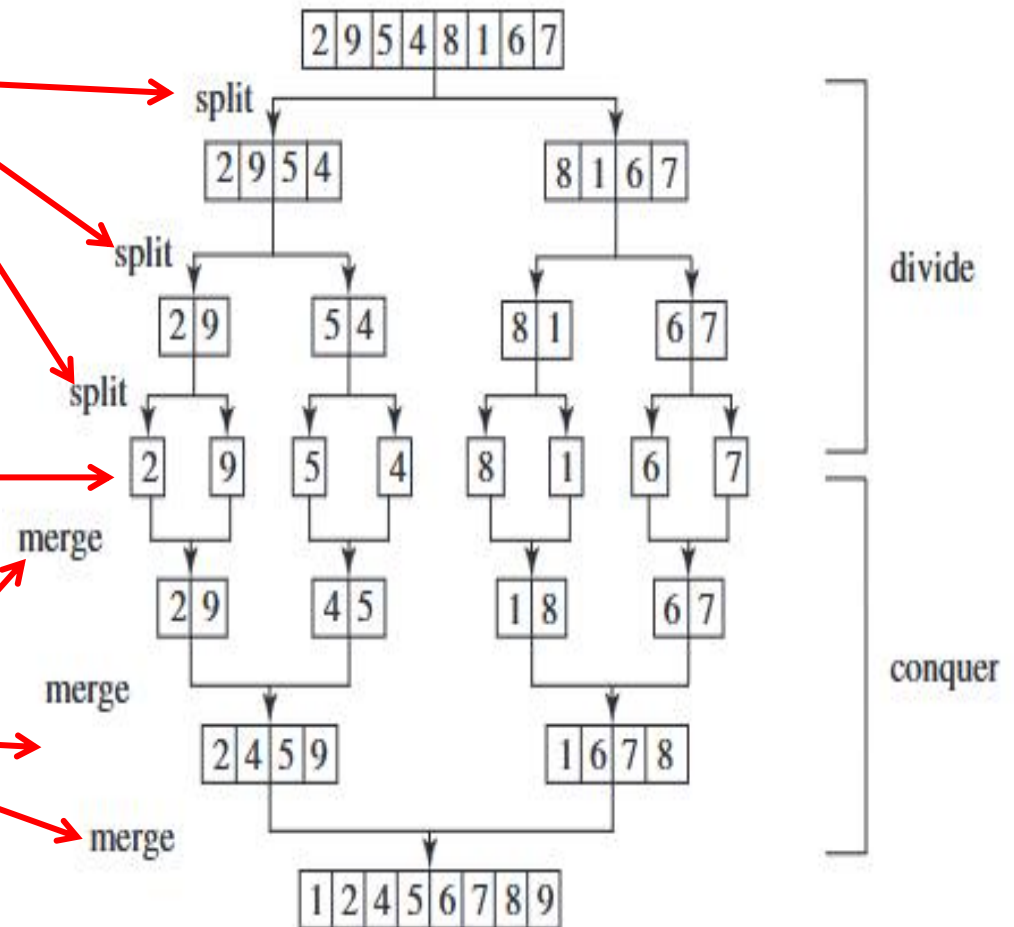
MergeSort()

// Keep dividing the list to halves by recursively calling itself

mergeSort(firstHalf);
mergeSort(secondHalf);

// stops at base case length=1
if (list.length > 1)

// As the recursive calls return,
merge() runs level by level to
sort and combine the sublists
merge(firstHalf, secondHalf, list);



Implementing in Main()

```
public static void main(String[] args) {  
    int size = 100000;  
    int[] a = new int[size];  
    randomInitiate(a);  
    long startTime = System.currentTimeMillis();  
    mergeSort(a);  
    long endTime = System.currentTimeMillis();  
    System.out.println((endTime - startTime) + "ms");  
}  
  
private static void randomInitiate(int[] a) {  
    for (int i = 0; i < a.length; i++)  
        a[i] = (int) (Math.random() * a.length);  
}  
  
}
```

16ms (v.s. 14650ms in bubble sort)

Merge Sort Time Complexity

In merge sort, the array is divided into two halves again and again:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow \dots \rightarrow 1$$

After k levels, the size of each subarray becomes $\frac{n}{2^k}$, so level $k = \log_2 n$

At each level, merging all subarrays processes all n elements once. This includes comparing elements and copying them into the temporary array, so the total work at each level is $O(n)$.

Therefore, the overall time complexity of Merge Sort is:

$$O(n) \times O(\log n) = O(n \log n)$$

Quick Sort

A quick sort works as follows:

- The algorithm selects an element, called the **pivot**, which is a reference value used to divide the array into two parts. The pivot can be any element in the array.
- It divide the array so that:
 - elements less than or equal to the pivot are placed on the left
 - elements greater than the pivot are placed on the right
- This process then happens **recursively** to the left and right parts until the array is sorted.



```

public static int partition(int[] list, int first, int last) {
    int pivot = list[first]; // Set the first element to be the pivot
    int low = first + 1;     // Set the low pointer position
    int high = last;         // Set the high pointer position

    // Case 1: before the two pointers met/crossed (step 1-5)
    while (high > low) {

        // Move low to the right until it stops at an element
        // greater than the pivot
        while (low <= high && list[low] <= pivot) {
            low++;
        }

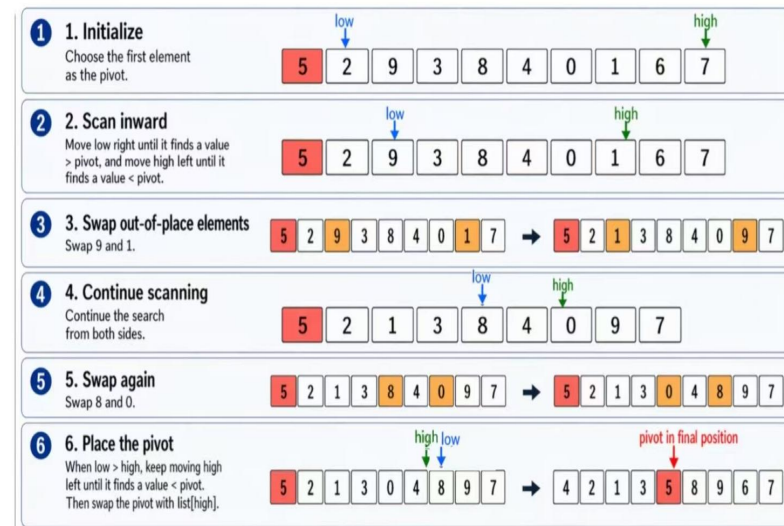
        // Move high to the left until it stops at an element
        // less than or equal to the pivot
        while (low <= high && list[high] > pivot) {
            high--;
        }

        // If the two pointers have not crossed, swap the two
        // elements
        if (high > low) {
            int temp = list[high];
            list[high] = list[low];
            list[low] = temp;
        }

        // Case 2: after the two pointers have met/crossed (step 6),
        // move high left until it finds an element smaller than the
        // pivot
        while (high > first && list[high] >= pivot) {
            high--;
        }

        // And then swap pivot with list[high]
        if (pivot > list[high]) {
            list[first] = list[high];
            list[high] = pivot;
            return high;
        } else { // Case 3: a sorted list
            return first;
        }
    }
}

```

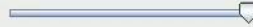


Array: 5,2,9,3,8,4,0,1,6,7

Start

Next Step

Partition Started Code Window Height:



pivot

high



low

Initial Array: [5, 2, 9, 3, 8, 4, 0, 1, 6, 7]

Pivot = 5 at index 0

low pointer = 1, high pointer = 9

```
int pivot = list[first];
int low = first + 1; // position, not value
int high = last; // position, not value
while (high > low) {
    while (low <= high && list[low] <= pivot) {
        low++;
    }
    while (low <= high && list[high] > pivot) {
        high--;
    }
    if (high > low) {
        int temp = list[high];
        list[high] = list[low];
        list[low] = temp;
    }
}
while (high > first && list[high] >= pivot) {
    high--;
}
if (pivot > list[high]) {
    list[first] = list[high];
    list[high] = pivot;
    return high;
} else {
    return first;
}
```

```

public class QuickSortTest {
    public static void quickSort(int[] list) {
        quickSort(list, 0, list.length - 1);
    }
    public static void quickSort(int[] list, int first,
int last) {
        if (last > first) {
            int pivotIndex = partition(list, first, last);
            quickSort(list, first, pivotIndex - 1);
            quickSort(list, pivotIndex + 1, last);
        }
    }
    public static int partition(int[] list, int first,
int last) {
        int pivot = list[first]; // Choose the first
        element as pivot
        int low = first + 1; // Index for forward search
        int high = last; // Index for backward search
        while (high > low) {
            while (low <= high && list[low] <= pivot)
                low++;

            while (low <= high && list[high] > pivot)
                high--;

            if (high > low) {
                int temp = list[high];
                list[high] = list[low];
                list[low] = temp;
            }
        }
        while (high > first && list[high] >= pivot)
            high--;
    }
}

```

This **quickSort()** method defines the recursive structure of Quick Sort.

Beacasue the **quickSort()** calls itself on the left subarray (list, first, pivotIndex-1) and right subarrays (list, pivotIndex+1, last) to keep dividing them.

In each recursive call, that is for each subarray, the **partition()** is used to separates the elements around the pivot and places the pivot in its correct position.

The recursion stops when a subarray has one element (**if (last > first)** statement fails, because, if one element, last==first).

```

    if (pivot > list[high]) {
        list[first] = list[high];
        list[high] = pivot;
        return high;
    } else
        return first;
}

public static void main(String[] args) {
    int size = 100000;
    int[] a = new int[size];
    randomInitiate(a);

    long startTime = System.currentTimeMillis();
    quickSort(a);
    long endTime = System.currentTimeMillis();

    System.out.println((endTime - startTime) + "ms");
}

private static void randomInitiate(int[] a) {
    for (int i = 0; i < a.length; i++)
        a[i] = (int) (Math.random() * a.length);
}
}

```

16ms (v.s. 16ms in merge sort and 14650ms in bubble sort)

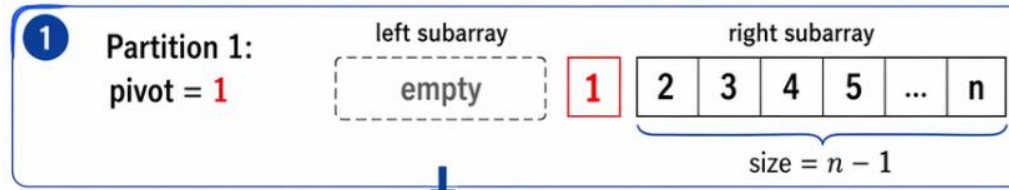
Quick Sort Time Complexity

- In the **best and average case**, each time the pivot will divide the array into 2 parts of the same or similar size. Simialr to the calculation in merge sort, the entire array can be divided k times where $k = \log_2 n$.
- At each level of division, the partition process use the pointers scans all n elements in total, so each level takes **$O(n)$** time.
- Therefore, the time complexity is

$$O(n) \times O(\log n) = O(n \log n)$$

Time Complexity - Worst Case

In the worst case, the pivot divides the array each time into **one big subarray while the other subarray is empty**. Example: $[1, 2, 3, 4, \dots, n]$, size = n



- In the 1st time partition, **pivot=1**, we have
 - left sub: [empty], right sub: $[2, 3, 4, 5 \dots n]$ with size = $n-1$
- In the 2nd time partition (*to divide $[2, 3, 4, 5 \dots n]$*), **pivot=2**,
 - left sub: [empty], right sub $[3, 4, 5, 6 \dots n]$ with size = $n-2$
- ... continues this way, we will have to recursively divide the array **$n-1$** times till the sub array size = 1
- Recall that the time complexity of each partition is **$O(n)$** , as it compares all elements during each division.
- So we did **$n-1$** times $O(n)$, so, we get the worst time complexity to be $n \cdot (n-1) = O(n^2)$

Heap Sort: Binary tree

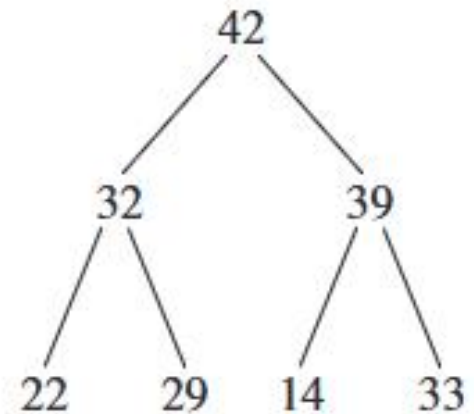
A **binary tree** is a hierarchical structure:

- It either is empty or it consists of an element, called the **root**.
- And two distinct binary trees, called the **left subtree** and **right subtree**
- The **length** of a path is the number of the edges in the path
- The **depth** of a node is the length of the path from the root to that node

For examples:

The length from 32 to 22: 1 (as there is only 1 edge)

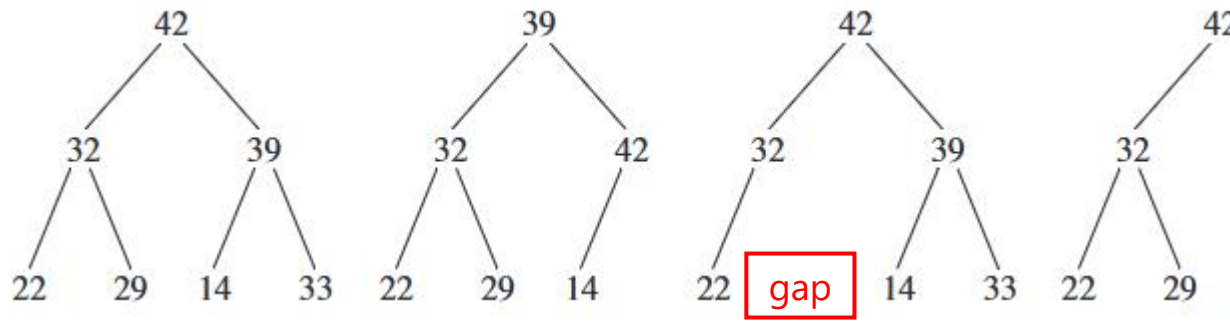
The depth of node 29: 2 (42(root)-32, 32-29)



Complete Binary Tree

A binary tree is *complete* if:

- All levels are completely.
- Sometimes, it is acceptable that the last level may not be completely full, BUT it must be filled from left to right, **without gaps**.

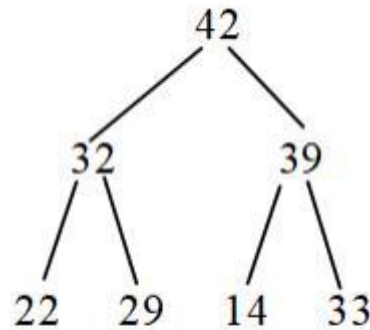


- 1st - **complete**, as all levels are completely filled
- 2nd - **complete**, although the last level is not full (missing one node next to 14), it is filled from left to right, without gaps
- 3rd, **incomplete**, a gap from left to right (next to node 22)
- 4th, **incomplete**, the 2nd level is not full, while we only allow the last level (level 3 in this case) not to be full

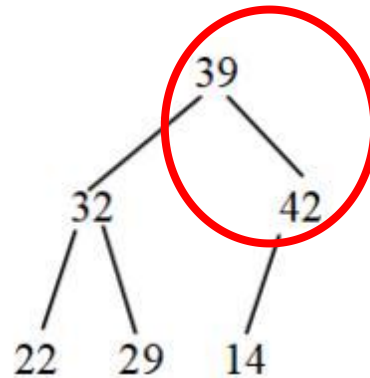
Binary Heap

- A *binary heap* is a binary tree with the following properties:
 - It is a **complete binary tree**, and
 - **Each node** is greater than or equal to any of its children

Example heap:



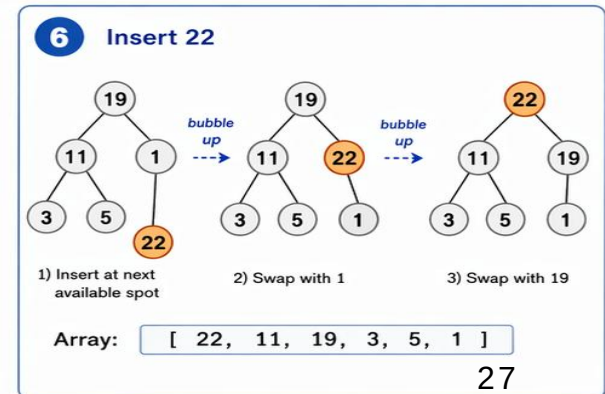
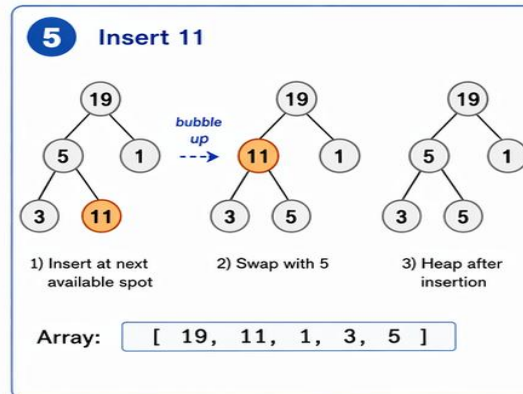
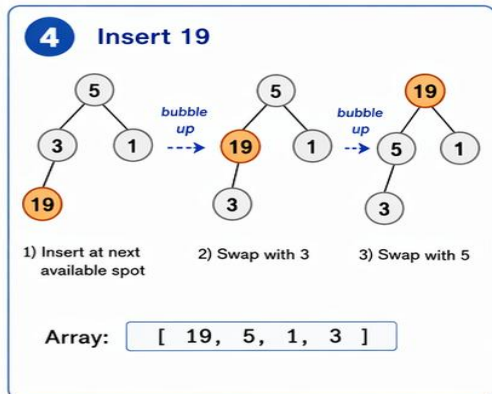
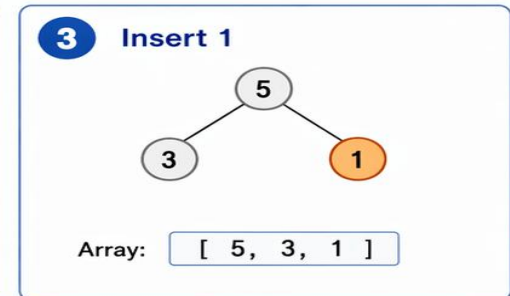
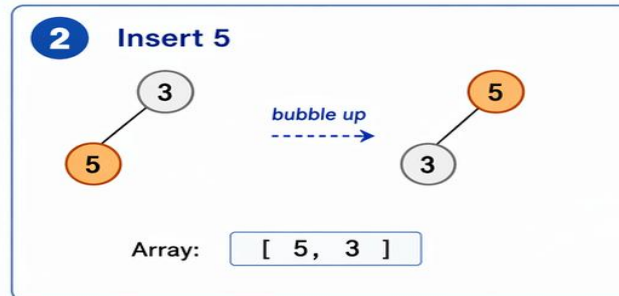
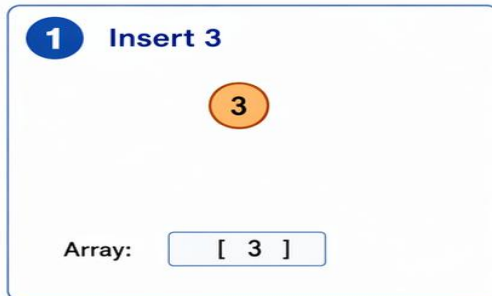
- Example not a heap, because the root (39) is less than its right child (42)



Adding Elements to a Heap

- One common operation in heap is to add new elements at the end of the heap.
 - Let the newly added node be the current node (the node we keep operating).
 - While it is greater than its parent:**
 - swap it with the parent, moving it one level up.
 - Stop when the current node is no longer greater than its parent, so the heap property is restored: each parent is greater than or equal to its children.

Insert in order: 3, 5, 1, 19, 11, 22

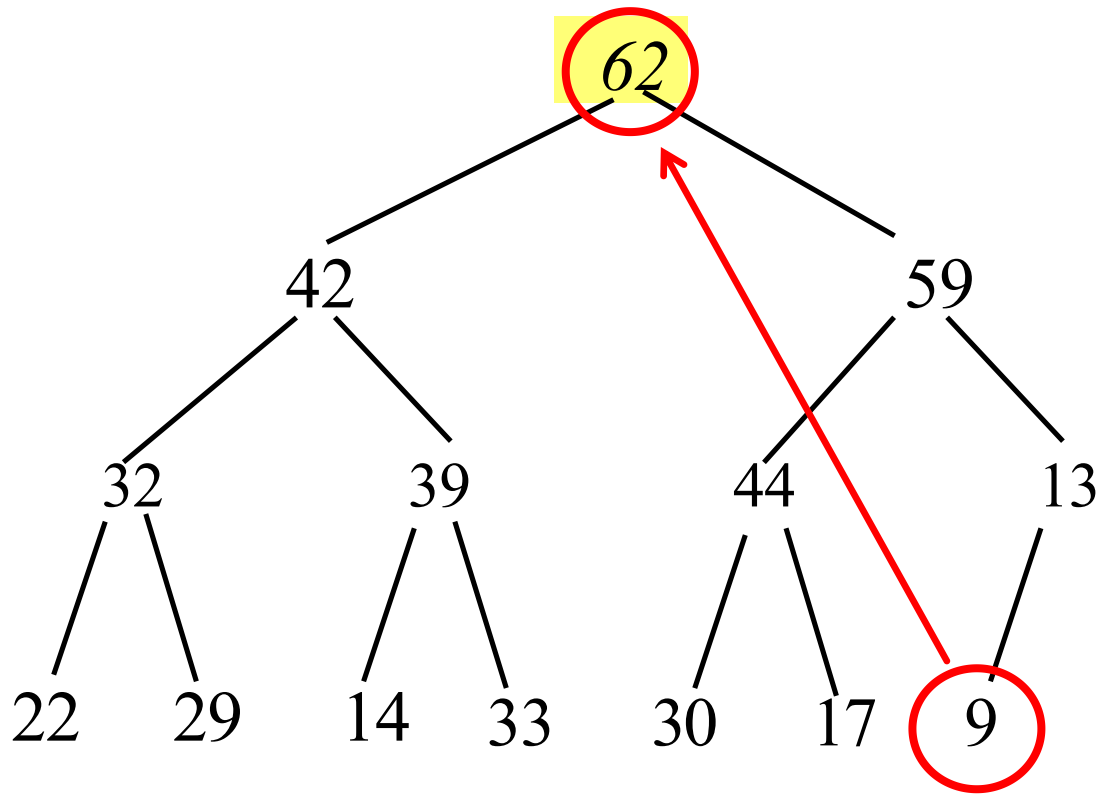


Removing the Root and Rebuild the Heap

- Another operation in heap is the removal. Often we need to remove the maximum element, which is the **root in a heap**
- After the root is removed, the tree must be rebuilt to **maintain the heap property (e.g., $\text{parent} \geq \text{child}$)**.

Removing the Root and Rebuild the Heap

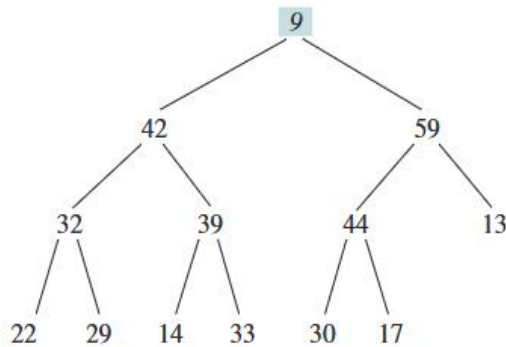
- Removing root 62 from the heap (replaces it with the last node in the heap: 9)



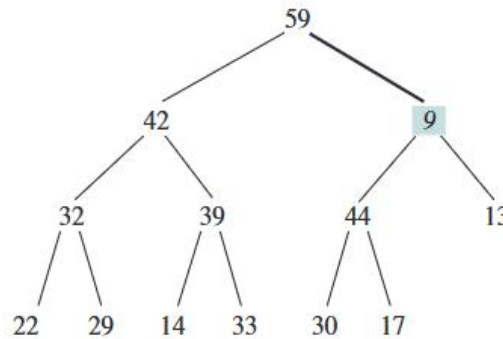
Move the last node to replace the root;
Let the root be the current node (the node we keep operating);

Removing the Root and Rebuild the Heap

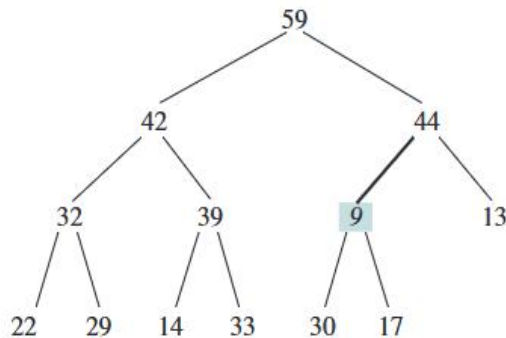
```
while (the current node has children and the current node < its children)
{
    Swap the current node with the larger of its children;
}
```



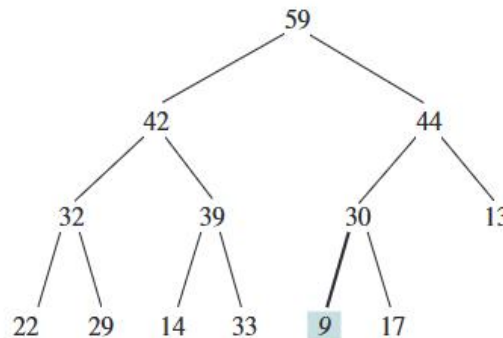
(a) After moving 9 to the root



(b) After swapping 9 with 59



(c) After swapping 9 with 44



(d) After swapping 9 with 30

(a-b) $9 < 42$; $9 < 59$; $59 > 42$;
So, swap 9 and 59
(because 59 is the larger child)

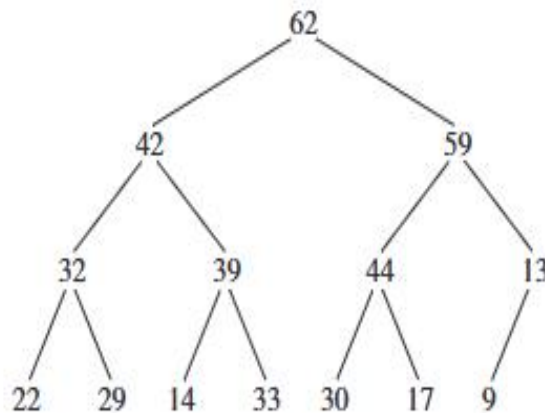
(b-c) $9 < 44$; $9 < 13$; $44 > 13$;
Swap 9 and 44

(c-d) $9 < 30$; $9 < 17$; $30 > 17$;
Swap 9 and 30

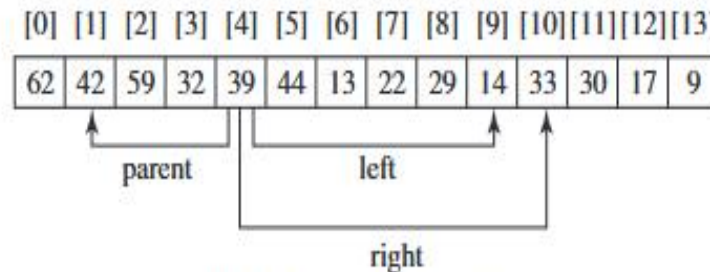
The heap is rebuilt now, and
ready for the next removal (to
remove 59)

Storing a Heap into Array

- A heap is usually shown as a binary tree, but it is often implemented using an array or an ArrayList..
- For a node at index i in an array, its left child can be stored at index $2i+1$ and its right child can be stored at $2i+2$, and its parent is at index $(i-1)/2$
 - For example: for a node at index 4 (value 39), its two children are at array index $2*4+1=9$ (value 14) and $2*4+2=10$ (value 33), and its parent is at index $(4-1)/2=1$, not 1.5 due to the rule of integer division, which is value 42.



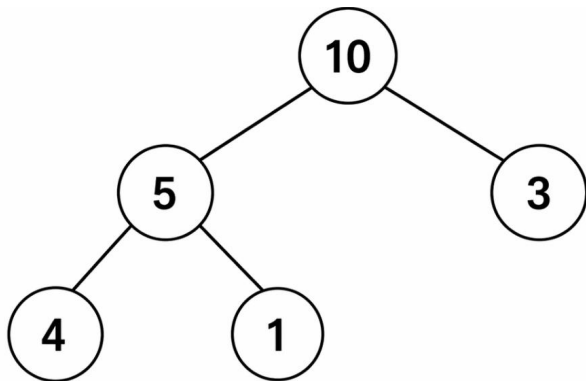
(a) A heap



(b) A heap stored in an array

Heap Sort

- **Now that we learnt:**
 - Reconstruction of a heap by adding and removing elements while maintaining its property (largest element at the root)
 - A heap can be stored efficiently in an array/ArrayList
- These properties make a **heap very suitable for sorting**.
 - Because, once a heap is (re)constructed, each time we remove the root, we get the current maximum element, and we can place it at the end of the same array.
 - We then get a sorted array from small to large.



For example:

- 1st removal: [..., 10]
- 2nd removal: [..., 5, 10]
- 3rd removal: [..., 4, 5, 10]
- (do removal repeatedly)
- Finally a sorted array: [1,3,4,5,10]

The Heap Class

Heap<E>
<pre>-list: java.util.ArrayList<E> -c: java.util.comparator<E></pre>
<pre>+Heap() +Heap(c: java.util.Comparator<E>) +Heap(objects: E[]) +add(newObject: E): void +remove(): E +getSize(): int +isEmpty(): boolean</pre>

Creates a default empty Heap.
Creates an empty heap with the specified comparator.
Creates a Heap with the specified objects.
Adds a new object to the heap.
Removes the root from the heap and returns it.
Returns the size of the heap.
Returns true if the heap is empty.

- The Heap class provided in our textbook offers the basic heap operations, such as insertion and removal of the root.
- These operations make it easier to implement and understand heap sort (also reflect the encapsulation principle!).

```

public class Heap<E extends Comparable> {
    private java.util.ArrayList<E> list = new java.util.ArrayList<E>();  /** Create a default
    heap */
    public Heap() {
    }
    /** Create a heap from an array of objects */
    public Heap(E[] objects) {
        for (int i = 0; i < objects.length; i++)
            add(objects[i]);
    }
    /** Add a new object into the heap */
    public void add(E newObject) {
        list.add(newObject); // Append to the end of the heap
        int currentIndex = list.size() - 1; // The index of the last node while (currentIndex >
        0) {

            int parentIndex = (currentIndex - 1) / 2;
            // Swap if the current object is greater than its parent
            if(list.get(currentIndex).compareTo(
                list.get(parentIndex)) > 0) {
                E temp = list.get(currentIndex);
                list.set(currentIndex, list.get(parentIndex));
                list.set(parentIndex, temp);
            } else
                break; // the tree is a heap now
            currentIndex = parentIndex;
        }
    }
}

```

```

/** Remove the root from the heap */
public E remove() {
    if (list.size() == 0) return null;

    E removedObject = list.get(0);
    list.set(0, list.get(list.size() - 1));
    list.remove(list.size() - 1);

    int currentIndex = 0;
    while (currentIndex < list.size()) {
        int leftChildIndex = 2 * currentIndex + 1;
        int rightChildIndex = 2 * currentIndex + 2;

        // Find the maximum between two children
        if (leftChildIndex >= list.size())
            break; // The tree is a heap
        int maxIndex = leftChildIndex;
        if (rightChildIndex < list.size())
            if (list.get(maxIndex).compareTo(
                list.get(rightChildIndex)) < 0)
                maxIndex = rightChildIndex;
    }
}

```

```

// Swap if the current node is less than the maximum
if (list.get(currentIndex).compareTo(
    list.get(maxIndex)) < 0) {
    E temp = list.get(maxIndex);
    list.set(maxIndex, list.get(currentIndex));
    list.set(currentIndex, temp);
    currentIndex = maxIndex;
}
else
    break; // The tree is a heap
}
return removedObject;
}

```

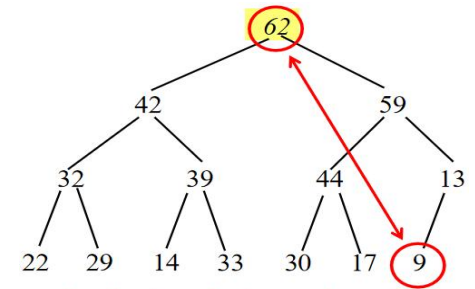
```

/** Get the number of nodes in the tree */
public int getSize() {
    return list.size();
}
}

```

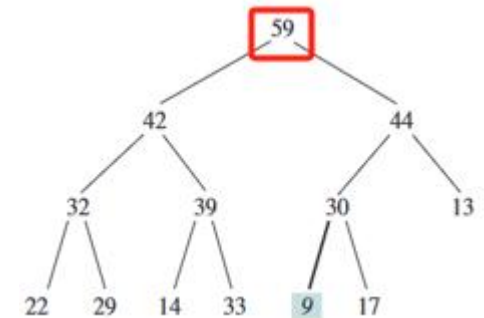
Heap Sort

```
public class HeapSort {  
    public static <E extends Comparable> void heapSort(E[] list) {  
        // Create a Heap of E  
        Heap<E> heap = new Heap<E>();  
  
        // Add elements to the heap  
        for (int i = 0; i < list.length; i++)  
            heap.add(list[i]);  
  
        // Remove the highest elements from the heap  
        // and store them in the list from end to start  
        for (int i = list.length - 1; i >= 0; i--)  
            list[i] = heap.remove();  
    }  
  
    /** A test method */  
    public static void main(String[] args) {  
        Integer[] list = {2, 3, 2, 5, 6, 1,  
            -2, 3, 14, 12};  
        heapSort(list);  
        for (int i = 0; i < list.length; i++)  
            System.out.print(list[i] + " ");  
    }  
}
```



1st removal

[, , , , 62]



2nd removal

[, , , 59, 62]



Storing from the largest one to the
smallest one 37

Heap Sort Time Complexity

Heap Sort Time: $O(n \log n)$

- In a heap, the nodes at different levels are:
 - Level 0: 1 node
 - Level 1: 2 nodes
 - Level 2: 4 nodes
 - ...
 - Level h : 2^h
 - So, the number of nodes is $1 + 2 + 4 + \dots + 2^h \approx 2^{h+1} - 1$, where the height $h \approx \log n$
 - When we add or remove an element, we only move it up or down one level at a time, so the number of steps is at most the height of the heap, which is about $\log n$.
 - If we have n elements, the time complexity is then $n * \log n = O(n \log n)$
- **Space Complexity (a dimension worth analyzing in report)**
 - Both merge and heap sorts require $O(n \log n)$ time.
 - A merge sort requires a temporary array for merging two subarrays; a heap sort does not need additional array space.
 - Therefore, a heap sort is more **space efficient** than a merge sort.

Life-Long Learning and EDI Principles in OOP

CPT 204 - Advanced OO
Programming

AY 25/26

Life-Long Learning in Object-Oriented Programming

- **Definition:**

- **Life-long learning** refers to the continuous, voluntary, and self-motivated pursuit of knowledge for personal or professional development.

- **Importance:**

- **Stay current:** Keep up with new frameworks, tools, and design patterns.
- **Build stronger skills:** Improve software design, coding, and maintenance.
- **Grow professionally:** Adapt to change and expand career opportunities

Ways to Keep Life-long Learning

- ***Maintain a reflective journal/log where you document:***
 - Key takeaways from each learning session.
 - Insights and thoughts gained after solving coding challenges.
 - Questions or areas where you need further clarification.
- ***Regular Self-Assessment:***
 - Review your progress against your learning goals,
 - Identify concepts you still find difficult,
 - Recognize the progress you have made.
- ***Join Communities and Forums***
 - Join OOP-focused communities on platforms like StackOverflow, Reddit, or GitHub.
 - Participate in discussions, ask questions, and share your knowledge with others.

EDI Principles in OOP

Equality, Diversity, and Inclusion (EDI) are essential principles that promote fairness, representation, and a welcoming environment within the software development community.

- **Equality** in OOP means ensuring that all developers, regardless of their gender, race, or disability, have equal access to career opportunities and the ability to contribute to the software development process.
- **Diversity** in OOP enriches problem-solving by bringing together people from various backgrounds, experiences, and perspectives. This diversity leads to more creative and innovative solutions to complex software design challenges.
- **Inclusion** in OOP goes beyond representation and equality; it emphasizes creating an environment where everyone feels respected and valued.

Practical Ways to Incorporate EDI into OOP Practices

Adopt Inclusive Coding Practices:

- Use inclusive language in code, comments, and documentation. Avoid terms or practices that could inadvertently alienate or exclude individuals.

Implement Accessible Design Patterns:

- Ensure that the software designs you build are accessible to a wide audience. Incorporate accessibility features such as customizable UI options, voice recognition, and high-contrast themes.

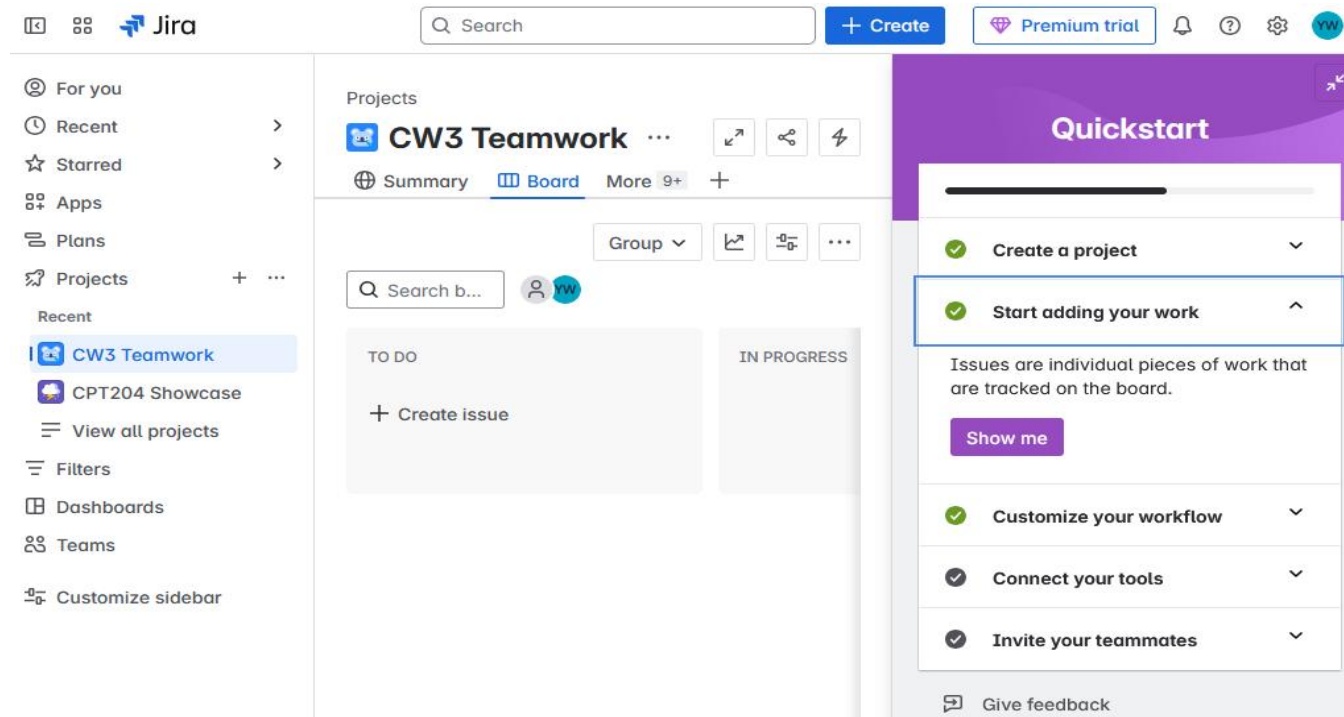
Encourage Collaboration Across Diverse Teams:

- Actively seek out team members from diverse backgrounds and experiences. This could mean promoting women in tech, or working with teams that include people from different ethnicities and cultures.

To answer questions in Group Project Task D, like:

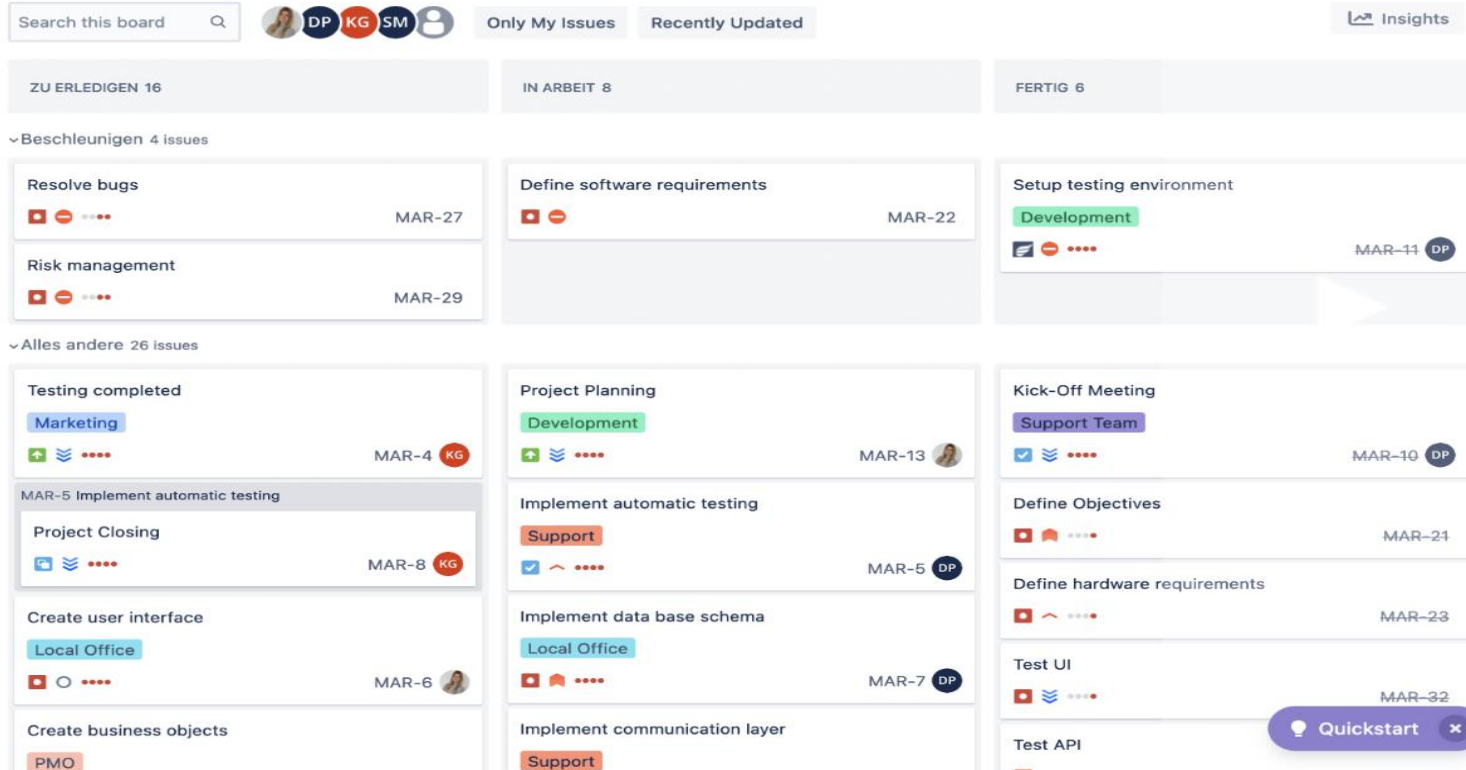
- *What is your understanding about equality, diversity, and inclusion in the context of this project?*
- *Why are these principles important in software design?*

Development with the AI-assisted Management Tools



- Develop advanced software components as a member or a leader of a software development team, **incorporating AI-assisted features.** (LO-C)
- Taking JIRA as an example, go through the **QuickStart** step by step.
- You can find JIRA here: <https://www.atlassian.com/software/jira>
- You may use other AI-assisted management tools (e.g., PingCode).

Development with the AI-assisted Management Tools



- **Basic Requirements for the Group Project (Task D):**
 - Show your planning board like the figure above
 - Make sure it is detailed (e.g., task date, allocation, date, etc)
 - Screenshots+descriptions should be good for the report.